Planificación de Procesos

Sistemas Operativos II

Licenciatura en Ciencias de la Computación
FCEIA - UNR

8 de abril de 2026

Notas

Los ejemplos correspondientes a los métodos de planificación se encuentran en la carpeta Ejemplos del Campus Virtual de la asignatura.

Planificación de procesos

Definición

Planificación de procesos es el mecanismo mediante el cual un sistema operativo gestiona la ejecución de los procesos, determinando el orden y la duración con que estos acceden al procesador y otros recursos del sistema.

Objetivo general

El objetivo general es optimizar el rendimiento general del sistema, asegurando equidad, eficiencia y respuesta adecuada a las necesidades de los procesos y usuarios.

Diagrama de transición entre los estados de un proceso

Modelo de cinco estados

Tipos de planificación

Hay tres tipos principales de planificación:

Largo plazo

El planificador a largo plazo se encarga de admitir nuevos procesos, es decir, de la transición de **nuevo** a **listo**.

Mediano plazo

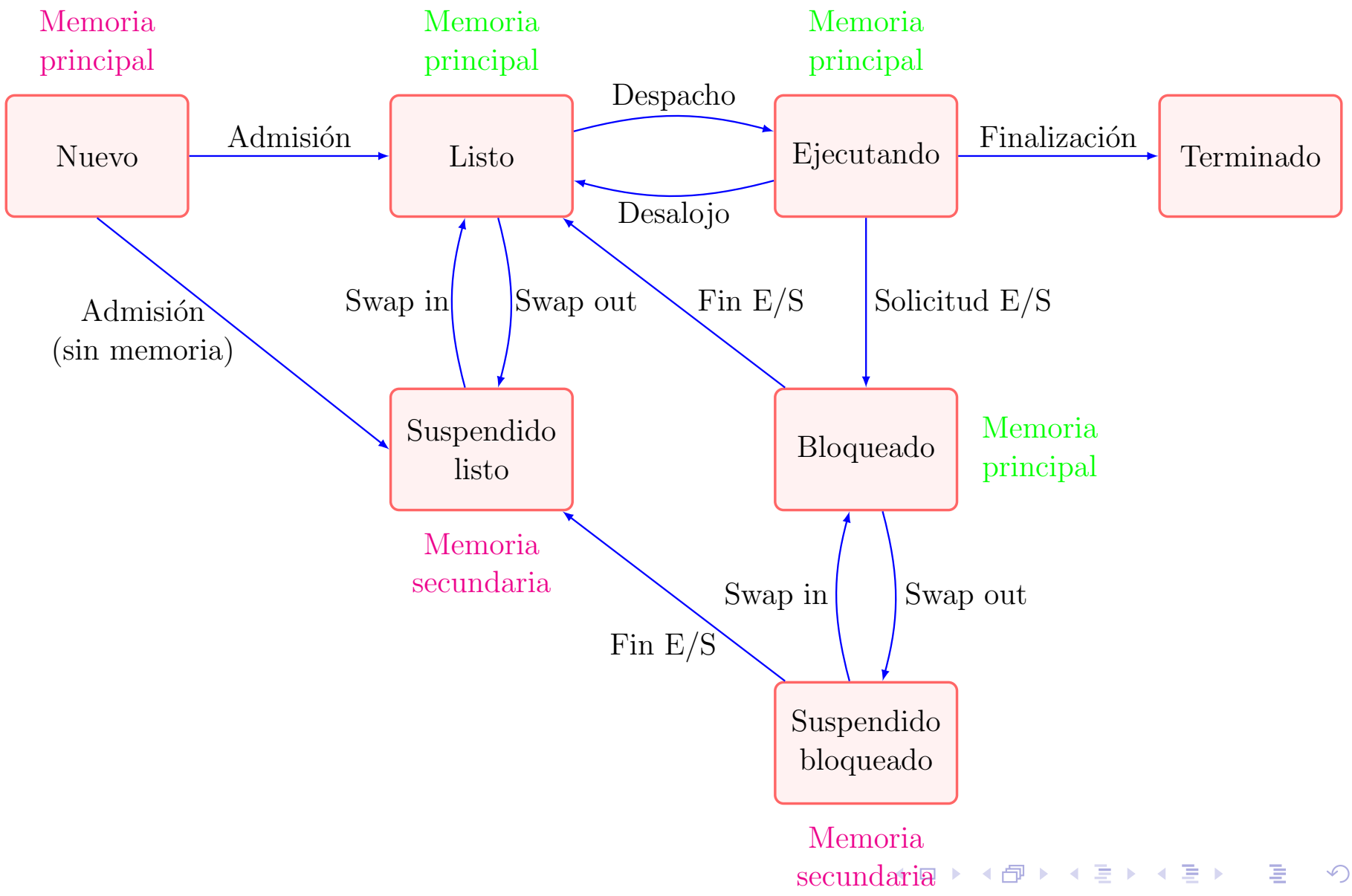
El planificador a mediano plazo gestiona la suspensión y reanudación de procesos (swapping), controlando el grado de multiprogramación mediante transiciones entre **listo** y **suspendido**, y entre **bloqueado** y **suspendido**.

Corto plazo

El planificador a corto plazo selecciona, entre los procesos en estado **listo**, el siguiente proceso a ejecutar.

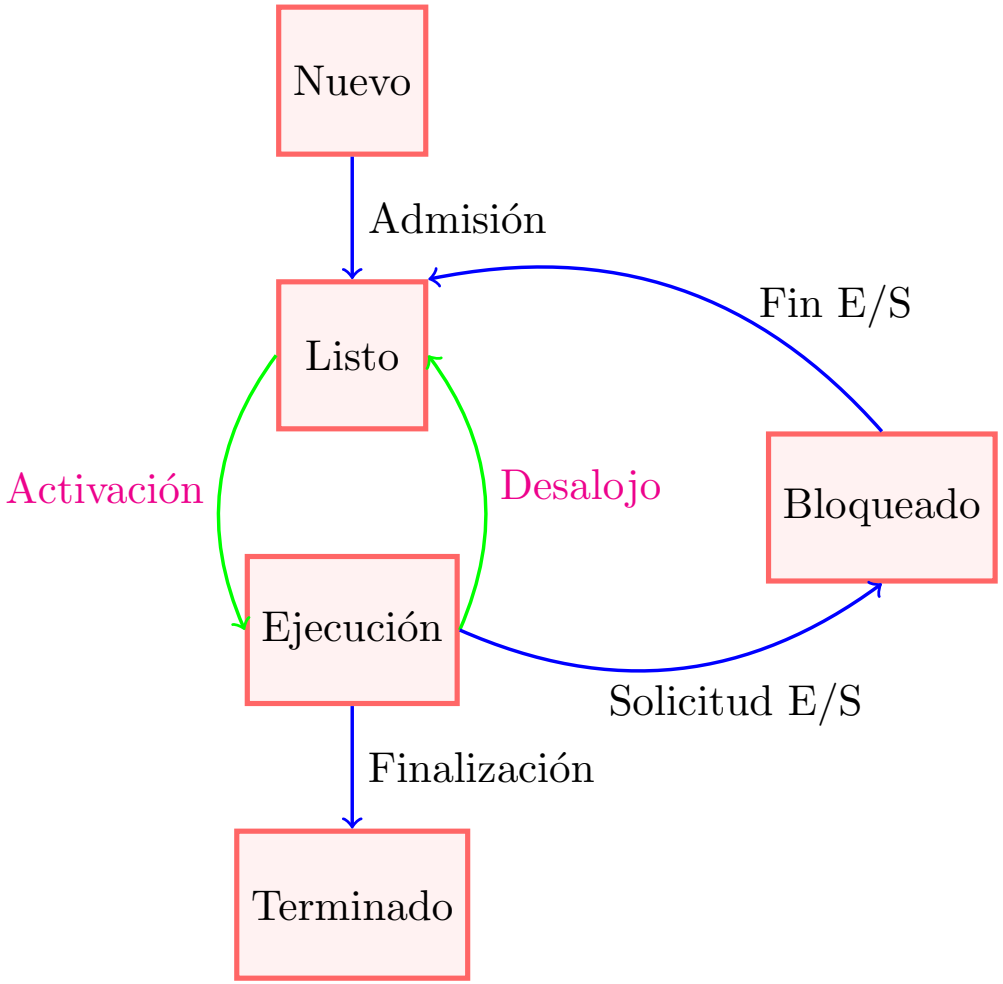
Diagrama de transición entre los estados de un proceso

Modelo de siete estados



Planificador de corto plazo

Abordaremos los diferentes tipos de planificadores a corto plazo, es decir, los mecanismos para gestionar las transiciones entre los estados de Listo y en Ejecución.

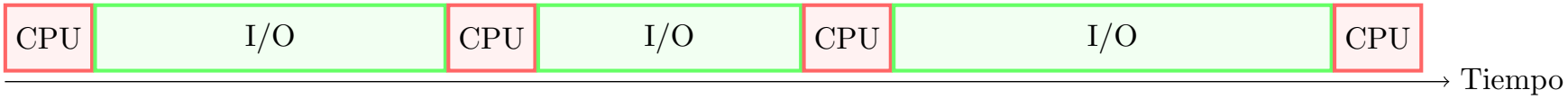


Tipos de procesos

Los procesos pueden separarse en dos grandes categorías:

- Procesos largos: procesos con alto tiempo total de ejecución, típicamente asociados a ráfagas de CPU largas (CPU-bound).

- Procesos cortos: procesos con bajo tiempo total de ejecución (ráfagas de CPU breves), típicamente asociados a tareas interactivas.



En general, se procura dar un tratamiento preferente a los procesos cortos, especialmente a los interactivos.

Tipos de procesos: ejemplos

- **Procesos interactivos (típicamente cortos):**
  - Editor de texto (por ejemplo, guardar un archivo o procesar una tecla).
  - Shell o terminal al ejecutar un comandos simples.
  - Aplicación gráfica que responde a un clic del usuario.
  - Navegador web al procesar un clic o desplazamiento.

  En estos casos, si el sistema responde rápido, el usuario percibe que el sistema es fluido.

- **Procesos de cálculo intensivo (típicamente largos):**
  - Compilación de un programa grande.
  - Simulación científica.
  - Renderizado de una imagen o video.
  - Procesamiento masivo de datos.

  Estos procesos pueden esperar más tiempo sin afectar demasiado la experiencia del usuario.

Midiendo la respuesta

Tick (definición)

- Un tick es un evento periódico generado por el temporizador del sistema que produce una interrupción de reloj, permitiendo al sistema operativo medir el paso del tiempo y ejecutar tareas de gestión, como la planificación y la contabilidad de procesos.
- En Linux, el parámetro `HZ` define la frecuencia de ticks (número de ticks por segundo); por ejemplo, si `HZ = 1000`, el intervalo entre ticks es aproximadamente 1 ms:

```
grep 'CONFIG_HZ=' /boot/config-$(uname -r)
```

Midiendo la respuesta

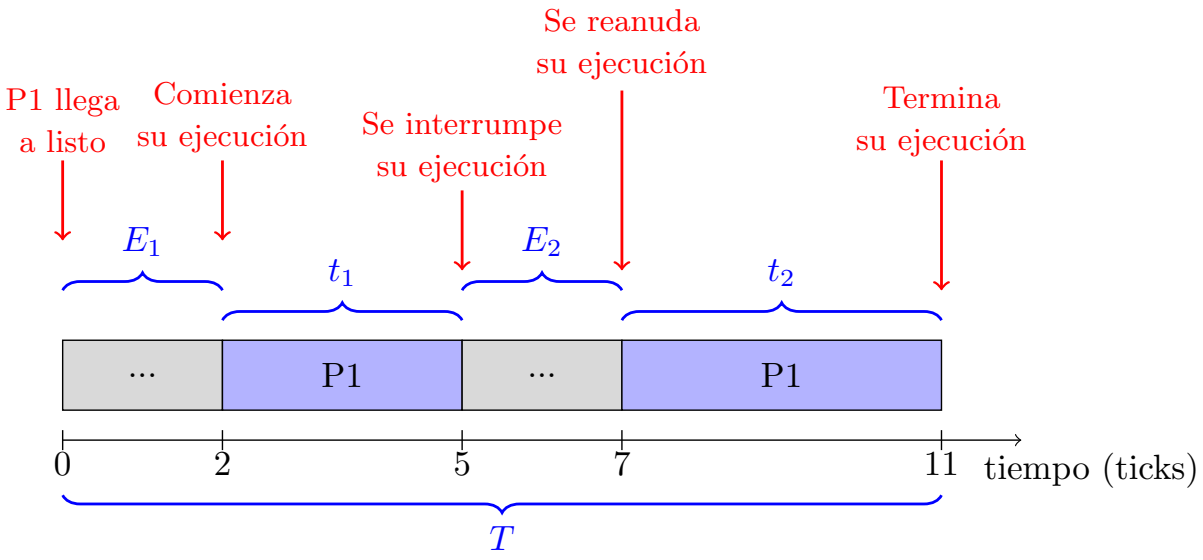
Quantum (definición)

- Un quantum es el intervalo máximo de tiempo de CPU que el planificador asigna a un proceso antes de poder interrumpirlo.
- Se utiliza en algoritmos apropiativos basados en reparto temporal (por ejemplo, Round Robin), mientras que no resulta relevante en planificación no apropiativa.
- El quantum suele expresarse en múltiplos del tick del temporizador del sistema, y su valor depende principalmente de la política de planificación utilizada.

Métricas

- Tiempo de ejecución (t): Tiempo total durante el cual el proceso se encuentra ejecutándose en la CPU.
- Tiempo de retorno (*turnaround time*) (T): Tiempo total transcurrido desde que el proceso se encuentra listo para ejecutarse hasta que finaliza, incluyendo el tiempo que permanece esperando la CPU.
- Tiempo de respuesta: Para un proceso interactivo, es el tiempo que trascurre desde que se lanza una petición hasta que se comienza a recibir la respuesta.
- Tiempo en espera ($E = T - t$): Es el tiempo durante el cual el proceso permanece listo, pero esperando la CPU.
- Proporción de penalización ($P = T/t$): Relación entre el tiempo de retorno y el tiempo de uso del procesador, que indica en qué proporción el proceso fue penalizado.
- Proporción de respuesta ($R = t/T$): Inversa de P.

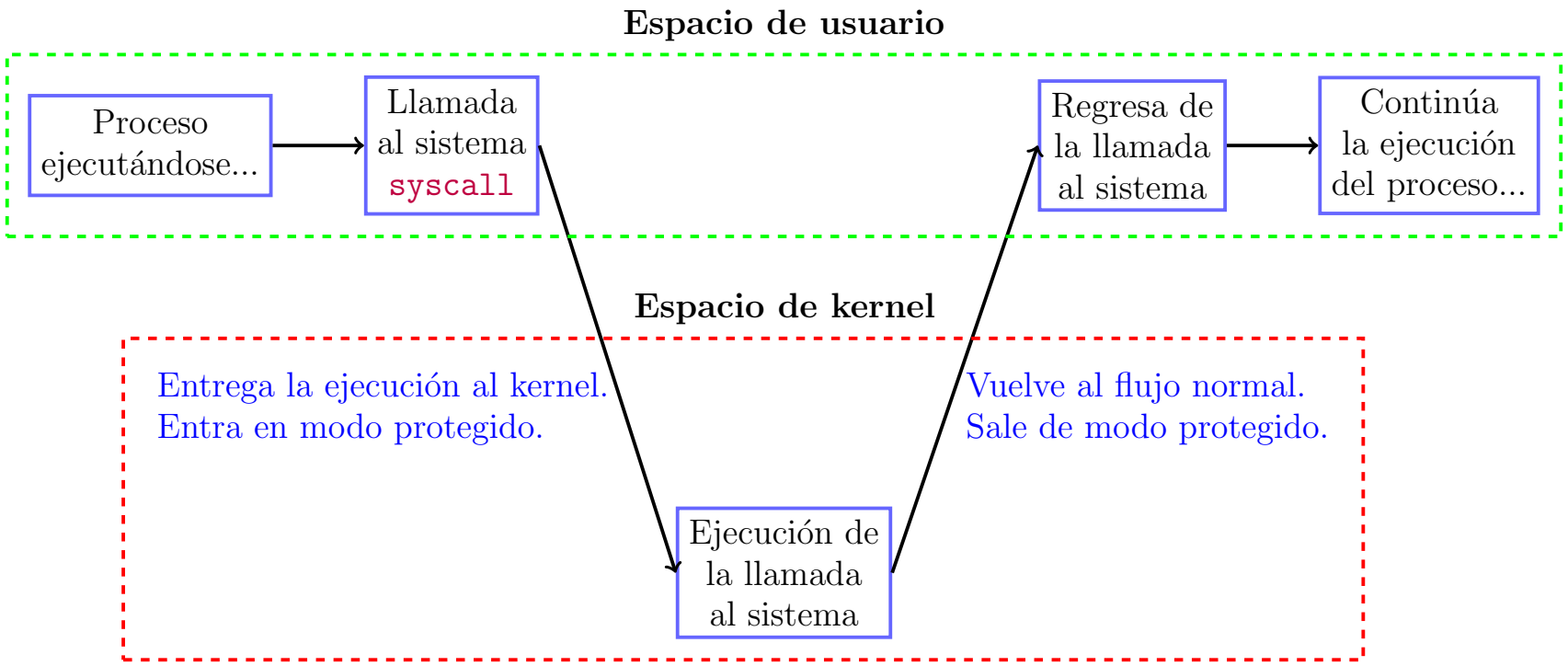
Métricas (Ejemplo)



- Tiempo de ejecución: $t = t_1 + t_2 = 3 + 4 = 7$
- Tiempo de espera: $E = E_1 + E_2 = 2 + 2 = 4$
- Tiempo de retorno (*turnaround time*): $T = 11$
- Proporción de penalización: $P = T/t = 11/7 = 1{,}6$
- Proporción de respuesta: $R = t/T = 7/11 = 0{,}6$

Tiempo de usuario vs. tiempo de sistema

- Un proceso se ejecuta alternando entre dos modos:
  - Modo usuario: ejecución del código propio del programa.
  - Modo sistema (kernel): ejecución de código del sistema operativo en respuesta a llamadas al sistema o interrupciones.
- Estos modos se reflejan en los tiempos: user, sys y real.



Tiempo de usuario vs. tiempo de sistema

- El comando `time` reporta:
  - `real`: tiempo de reloj/*wall-clock*
    Es el tiempo total transcurrido desde que se ejecuta el comando hasta que termina.
    - Incluye todo: CPU, esperas, bloqueos, I/O, planificación, etc.
    - Es lo que se mediría con un cronómetro.
  - `user`: tiempo en modo usuario
    Es el tiempo que la CPU pasa ejecutando código del programa en sí.
    - No incluye llamadas al sistema.
    - Representa trabajo "útil" del programa.
  - `sys`: tiempo en modo kernel
    Es el tiempo que la CPU pasa ejecutando código del sistema operativo en nombre del proceso.
    - Ejemplos: llamadas a `read`, `write`, manejo de archivos, etc.
- `user + sys` mide uso de CPU, mientras que real mide tiempo de vida del proceso.
- Puede haber pequeñas diferencias porque hay cambios de contexto, interrupciones, resolución del reloj (no es perfecta), etc.

Tiempo de usuario vs. tiempo de sistema

Demo

Comparar los tiempos de ejecución de dos programas: uno CPU-bound (`cpu.c`) y otro I/O-bound (`io.c`).

```
$ time ./cpu
real    0m4.722s
user    0m4.701s
sys     0m0.001s

$ time ./io
real    0m10.043s
user    0m0.002s
sys     0m0.000s
```

El programa CPU-bound está limitado por la capacidad de cómputo (uso intensivo de CPU), mientras que el I/O-bound está limitado por la latencia de entrada/salida.

Objetivo general del planificador a corto plazo

- El objetivo general del planificador a corto plazo (despachador) es decidir qué proceso en estado **listo** debe ocupar la CPU.
- El despachador puede ser invocado cuando un proceso se encuentra en alguna de las siguientes cuatro circunstancias:
  1. Pasa de estar **ejecutando** a estar **bloqueado** (por ejemplo, al solicitar una operación de E/S, esperar la sincronización con otro proceso, etc.).
  2. Pasa de estar **ejecutando** a estar **listo** (por ejemplo, al ocurrir la interrupción del temporizador o algún evento externo).
  3. Pasa de estar **bloqueado** a estar **listo** (por ejemplo, al finalizar la operación de E/S solicitada).
  4. Finaliza su ejecución y pasa de **ejecutando** a **terminado**.
- La situación (2) implica planificación apropiativa. La situación (3) puede implicarla si el sistema es apropiativo. En (1) y (4), el proceso deja la CPU voluntariamente.

Objetivos particulares del planificador a corto plazo

Ser justo: Los procesos que compartan las mismas características deben recibir un trato equitativo, evitando que alguno de ellos sea postergado indefinidamente.

Maximizar el rendimiento: Maximizar la cantidad de procesos completados por unidad de tiempo.

Ser predecible: Un mismo trabajo debería completarse en un tiempo aproximado, independientemente de la carga del sistema.

Minimizar la sobrecarga: Minimizar el tiempo dedicado a la gestión del algoritmo, ya que disminuye el tiempo de CPU disponible para trabajo útil.

Equilibrar el uso de recursos: Favorecer los procesos que utilicen recursos subutilizados y desalentar aquellos que compitan por recursos sobreutilizados.

Objetivos particulares del planificador a corto plazo

Evitar la postergación indefinida: Aumentar gradualmente la prioridad de los procesos más antiguos para facilitar su ejecución.

Favorecer el uso esperado del sistema: En sistemas interactivos se priorizan los procesos que atienden solicitudes de los usuarios.

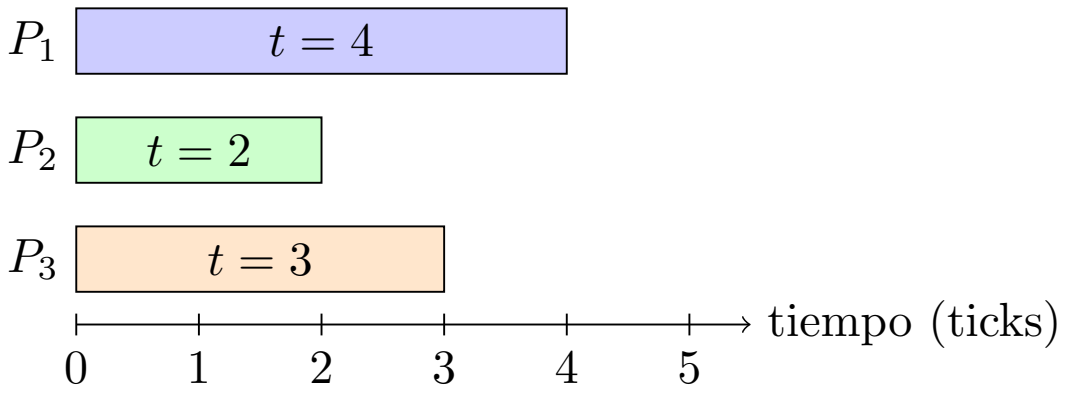
Evitar bloqueos: Si un proceso de baja prioridad posee un recurso requerido por otros, conviene favorecer su pronta finalización.

Favorecer comportamientos deseables: Penalizar procesos que consumen largos intervalos continuos de CPU.

Degradarse suavemente: Cuando aumenta la carga del sistema, el algoritmo debería minimizar el impacto sobre los procesos existentes.

Problema de planificación

Supongamos tres procesos con tiempos de ejecución de CPU conocidos:



- **¿En qué orden deberían ejecutarse?**
- La respuesta depende de los objetivos del sistema.
- Los algoritmos de planificación proponen distintas estrategias para resolver este problema.
- A continuación se presentan distintos métodos ordenados según su complejidad creciente.

Métodos de planificación de CPU

Planificación no apropiativa (non-preemptive)

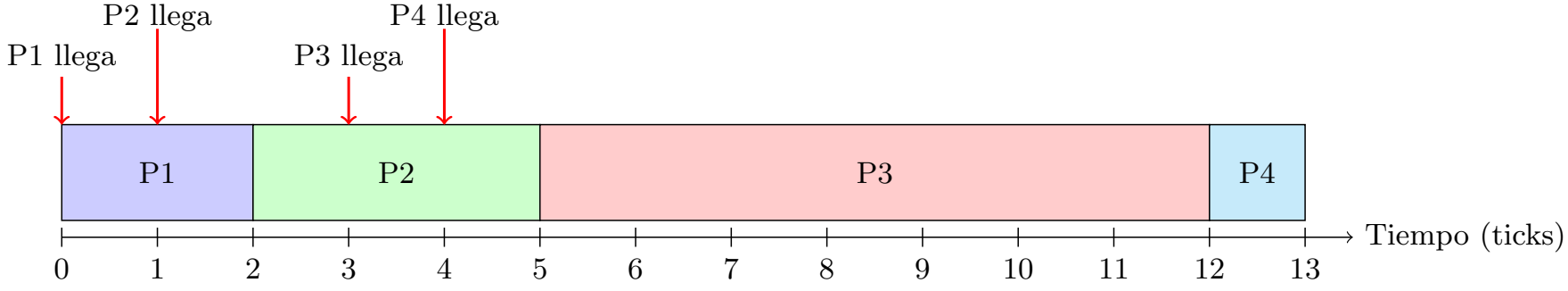
- Cada proceso mantiene la CPU hasta que finaliza, o se bloquea (por ejemplo, E/S).
- No hay interrupción forzada por parte del sistema operativo.
- **Ejemplos:** FCFS (*First-Come, First-Served*), SJF (*Shortest Job First*, versión no apropiativa), HPRN (Highest Penalty Ratio Next), etc.

Planificación apropiativa (preemptive)

- El sistema operativo puede quitar la CPU a un proceso.
- Requiere interrupciones (por ejemplo, al finalizar un quantum).
- Permite mejor respuesta en sistemas interactivos.
- **Ejemplos:** Round Robin, SRTF (*Shortest Remaining Time First*), planificación por prioridades apropiativa, etc.

Primero llegado, primero servido (FCFS, *First-Come, First-Served*)

- **FCFS** es un mecanismo no apropiativo con lógica mínima; su despachador es simple y se basa en una cola FIFO.
- Cada proceso se ejecuta en el orden de llegada y mantiene el control de la CPU hasta liberarlo.
- FCFS es poco adecuado para trabajo interactivo; su uso en sistemas modernos es limitado y se restringe a contextos específicos.
- Puede provocar el *"efecto convoy"* cuando un proceso largo bloquea a muchos procesos cortos.



El proceso más corto a continuación (SPN, *Shortest Process Next*)

- Si no es posible implementar **multitarea apropiativa** pero se desea un algoritmo más equitativo, puede seleccionarse el proceso más corto entre los que se encuentran en la cola de listos, siempre que se disponga de información previa sobre su tiempo de ejecución.
- En la práctica es difícil conocer esta información antes de ejecutar el proceso. Por ello, suele estimarse a partir de su historial de ejecución.
- **SPN** prioriza los procesos de menor duración, lo que mejora el tiempo de retorno promedio, aunque puede provocar esperas prolongadas en procesos largos.
- Puede provocar **postergación indefinida** (*Starvation*) de procesos largos.

El más penalizado a continuación (HPRN, *Highest Penalty Ratio Next*)

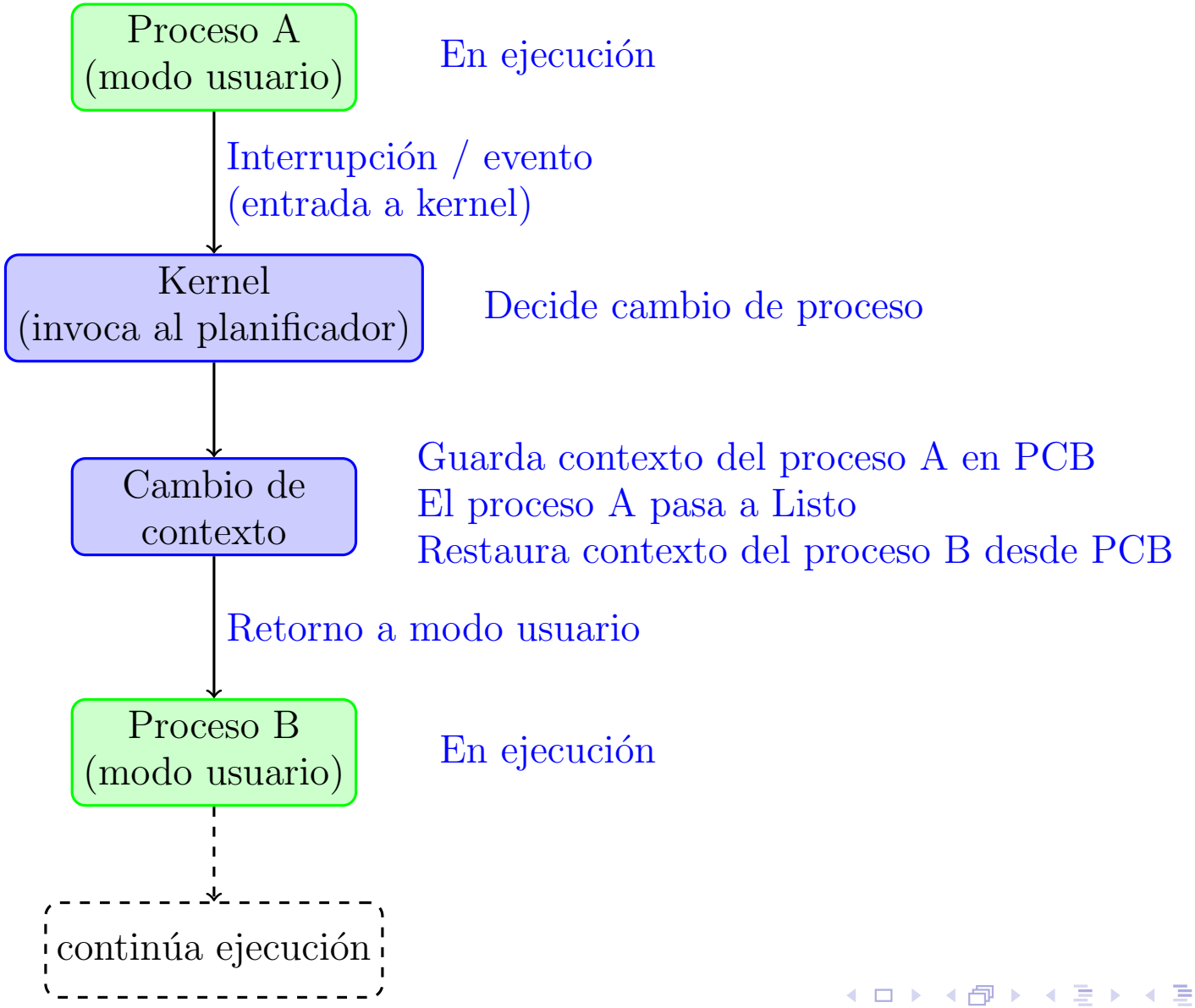
- Es un algoritmo **no apropiativo**, la selección del proceso se realiza solo cuando la CPU queda libre.
- Cada proceso entra a la cola de listos con un valor de penalización inicial $PR = 1$.
- PR se actualiza como $PR = \frac{w+t}{t}$.
- Se ejecuta el proceso con mayor valor de PR.
- Mientras la carga del sistema sea moderada, **HPRN** evita la inanición.
- **HPRN** se ubica conceptualmente entre **FCFS** y **SPN**: combina eficiencia (priorizar procsos cortos) con equidad (evitar espera indefinida).

Apropiación (preemption)

Es el mecanismo mediante el cual el sistema operativo le quita la CPU a un proceso en ejecución para asignársela a otro:

- **Activación del scheduler:** Por interrupciones o eventos del sistema.
- **Paso a modo kernel:** Cambio automático desde modo usuario.
- **Ejecución del scheduler:** Evalúa qué proceso debe ejecutarse a continuación según el algoritmo.
- **Decisión de apropiación:** Determina si se realiza un cambio de proceso.
- **Cambio de contexto:**
  - Guarda el contexto del proceso en ejecución.
  - El proceso pasa a estado listo.
  - Restaura el contexto del proceso seleccionado.
- **Retorno a modo usuario:** Se ejecuta el proceso seleccionado.

Apropiación (preemption)

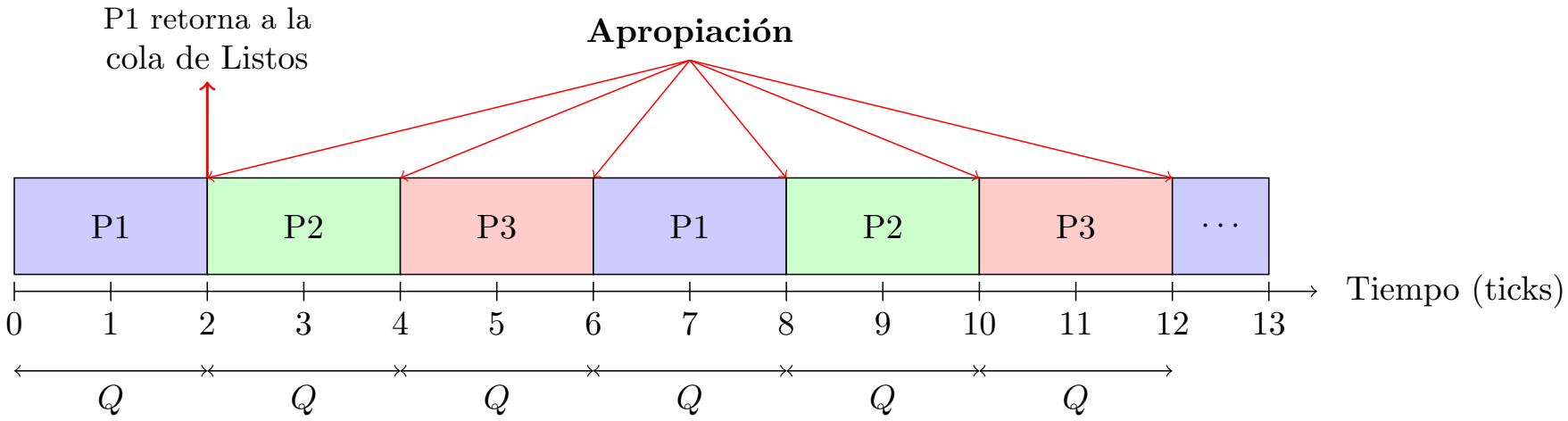


Ronda (*Round Robin*)

- Busca proporcionar un buen tiempo de respuesta tanto para procesos largos como cortos.
- Emplea **multitarea apropiativa**: mecanismo mediante el cual el sistema operativo puede interrumpir un proceso en ejecución y reasignar la CPU a otro, sin que el proceso en curso la ceda voluntariamente.
- Cada proceso dispone de un *quantum* de CPU por turno.
- Si no finaliza su ejecución al término de su *quantum*, es interrumpido y reubicado al final de la cola de **listos** para esperar nuevamente su turno.
- Los procesos que se desbloquean (por ejemplo, tras finalizar una operación de E/S) también se agregan al final de esta cola.

Ronda (*Round Robin*)

- Ejemplo con tres procesos y quantum $Q = 2$.
- El orden de llegada es P1, P2, P3.



Ronda (*Round Robin*)

- El comportamiento del algoritmo puede ajustarse modificando la duración del *quantum*:
  - Si el quantum es muy grande $\rightarrow$ FCFS.
  - Si el quantum es muy pequeño $\rightarrow$ aumenta la sobrecarga por cambios de contexto.
- **Round Robin** logra un compromiso entre equidad y tiempo de respuesta, garantizando que todos los procesos reciban CPU periódicamente.
- Sin embargo, su rendimiento depende fuertemente de la elección del quantum.

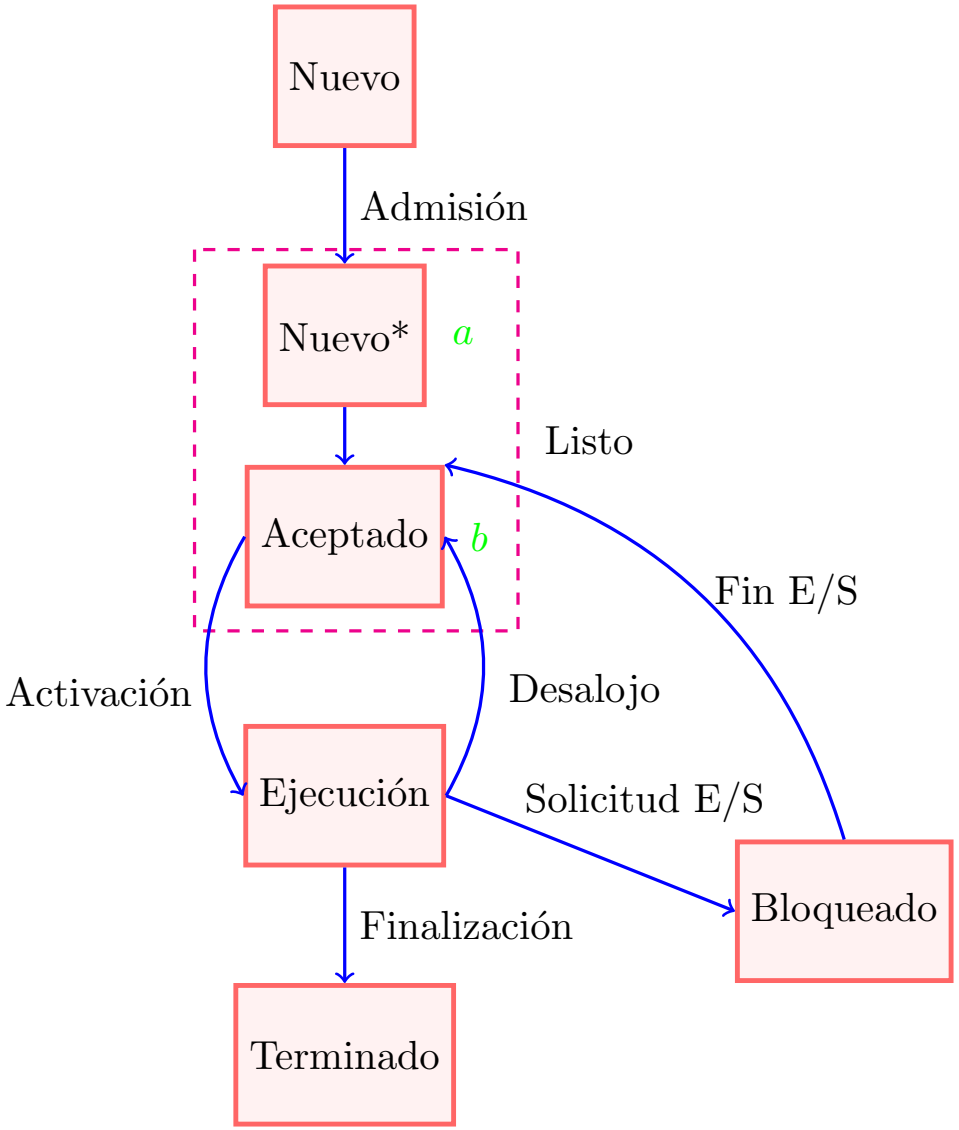
SPN apropiativo (PSPN, *Preemptive Shortest Process Next*)

- **PSPN** combina las estrategias de **SPN** con un esquema de **multitarea apropiativa**.
- También se lo conoce como ***Shortest Remaining Time*** **(SRT)**.
- Los procesos se incorporan a la cola de listos a medida que solicitan el uso de la CPU.
- El proceso con el menor tiempo de ejecución de CPU restante inicia (o continúa) la ejecución.
- Si llega otro proceso con un tiempo de ejecución de CPU restante menor, el proceso en ejecución se interrumpe y la CPU se asigna al nuevo proceso más corto.
- Minimiza el tiempo de espera promedio, pero requiere estimar la duración de los tiempos de ejecución de CPU.

Ronda egoísta (SRR, *Selfish Round Robin*)

- En el método selfish round-robin cada proceso tiene una prioridad dinámica y el planificador siempre ejecuta los procesos con mayor prioridad.
- Entre procesos con igual prioridad se usa Round Robin clásico.
- La cola de listos se divide en dos subcolas: **Nuevos*** y **Aceptados**.
- La prioridad de un proceso nuevo aumenta a una tasa a, mientras que la prioridad de un proceso aceptado aumenta a una tasa b.
- Los procesos nuevos no se programan directamente, sino que permanecen en la subcola de procesos **nuevos***. Solo se avanza con la subcola de procesos **aceptados**.
- Cuando la prioridad de un proceso nuevo alcanza la de un proceso aceptado, el nuevo se mueve a la subcola de **aceptados**.
- Si la subcola de procesos aceptados queda vacía, se acepta el proceso nuevo de mayor prioridad.

Ronda egoísta (SRR, *selfish round robin*)



Ronda egoísta (SRR, *Selfish Round Robin*)

- Si $\frac{b}{a} < 1$, la prioridad de un proceso recién llegado eventualmente alcanza la de los procesos aceptados y comienza a ejecutarse.
- Si $\frac{b}{a} = 1$, el proceso en ejecución termina y el siguiente proceso se acepta. En este caso, el esquema se comporta como **FCFS**.
- Si $\frac{b}{a} = 0$ (es decir, $b = 0$), los procesos recién llegados se aceptan inmediatamente y el algoritmo se comporta como una ronda (**RR**).
- **Conclusión:** Si $0 < \frac{b}{a} < 1$, la ronda es relativamente egoísta: los procesos nuevos ingresan incluso si los que llevan más tiempo ejecutándose son muy largos.

Ronda egoísta (SRR, *Selfish Round Robin*)

Ventajas:

- Evita postergación indefinida (*starvation*) de los procesos nuevos.
- Favorece los procesos largos sin bloquear completamente los cortos.
- Permite ajustar el comportamiento variando la relación b/a.

Desventajas:

- Introduce complejidad y requiere ajuste fino de parámetros, pudiendo afectar la equidad y la previsibilidad del sistema.

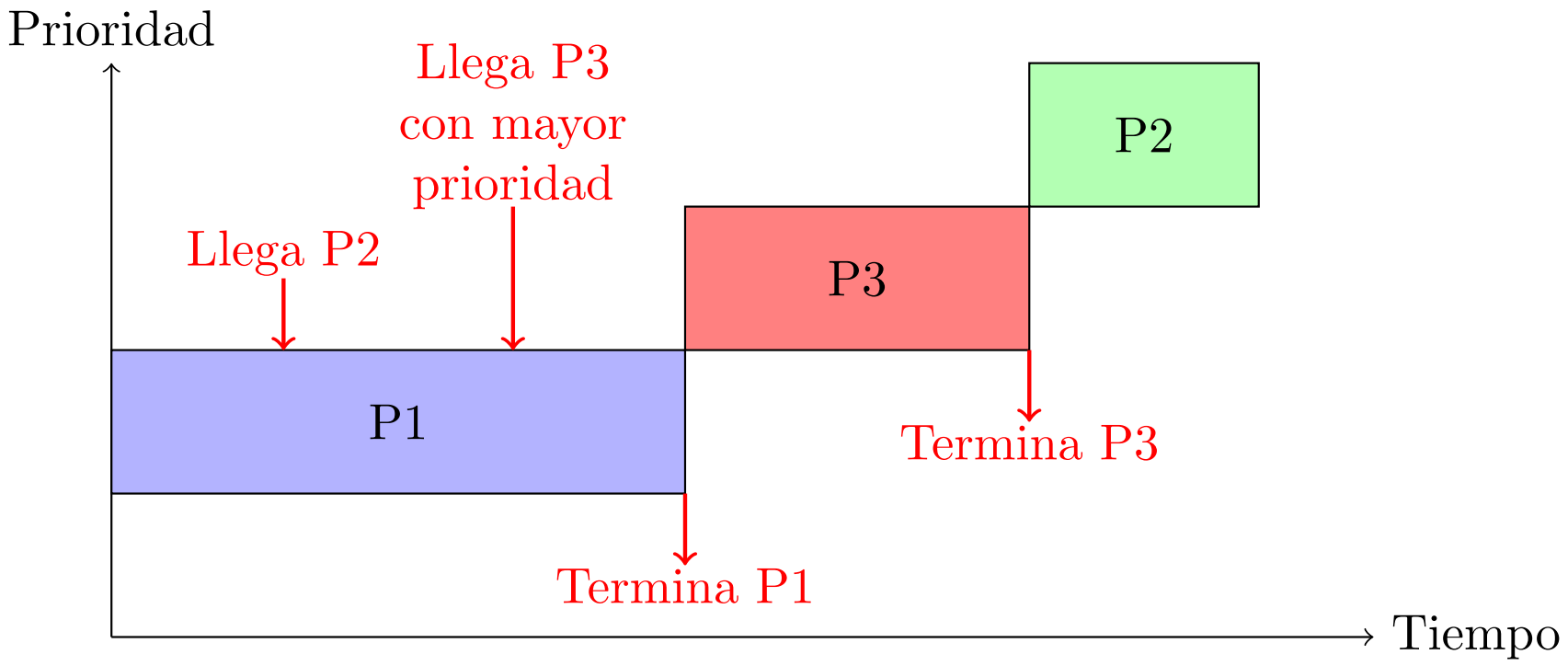
Conclusión: Favorece los procesos que ya llevan tiempo en ejecución ("*selfish*"), en lugar de los recién llegados.

Planificación por prioridades

- A cada proceso se le asigna una prioridad, y el planificador siempre selecciona el de mayor prioridad.
- Las prioridades pueden ser **internas** o **externas**:
  - **Prioridades internas**: basadas en características del proceso.
  - **Prioridades externas**: definidas por el usuario o el sistema.
- Las prioridades pueden ser **estáticas** o **dinámicas**:
  - **Prioridades estáticas**: cada proceso mantiene la misma prioridad durante toda su ejecución.
  - **Prioridades dinámicas**: las prioridades pueden cambiar durante la ejecución según políticas del planificador.

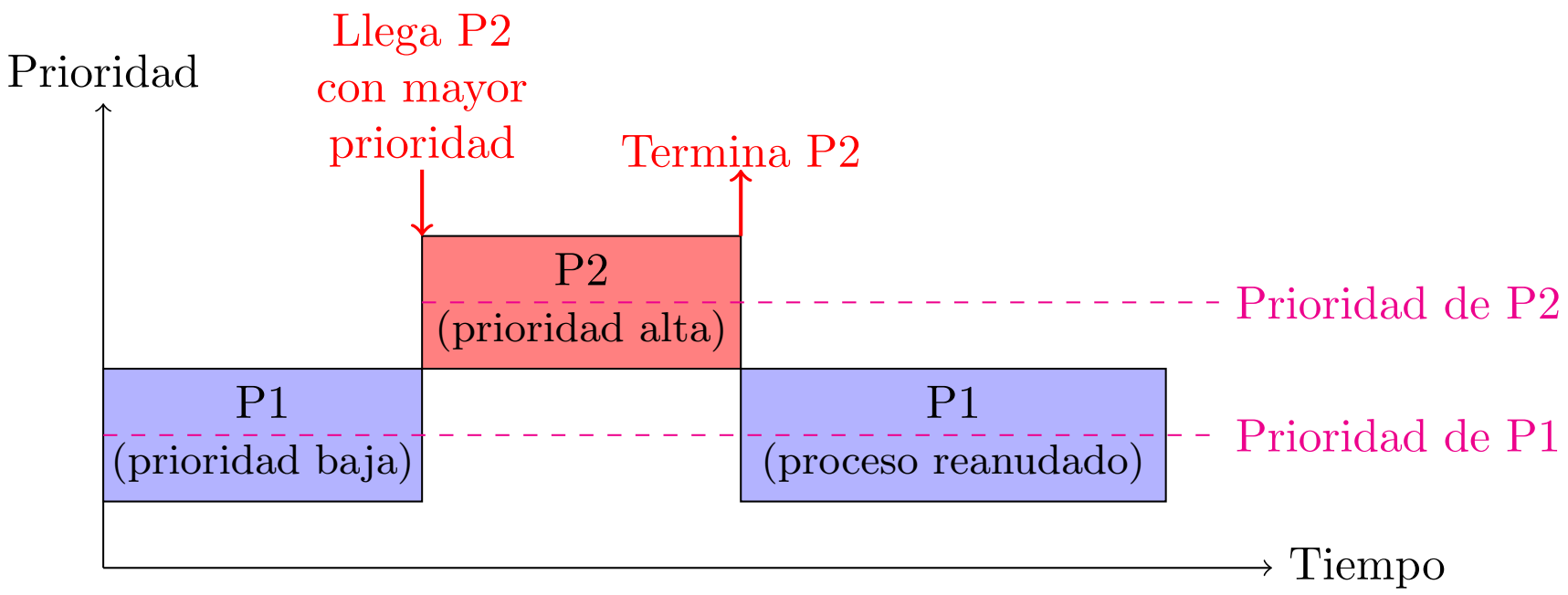
Planificación por prioridades (versión no apropiativa)

- La versión más simple de la planificación por prioridades es **no apropiativa**.
- En este caso, un proceso en ejecución no es interrumpido y las prioridades se evalúan solo al finalizar o bloquearse.



Planificación por prioridades (versión apropiativa)

- El método de planificación por prioridades puede ser apropiativo.
- Si el sistema utiliza planificación **apropiativa**, un proceso en ejecución es interrumpido cuando llega (o se vuelve listo) un proceso con mayor prioridad.



Planificación por prioridades

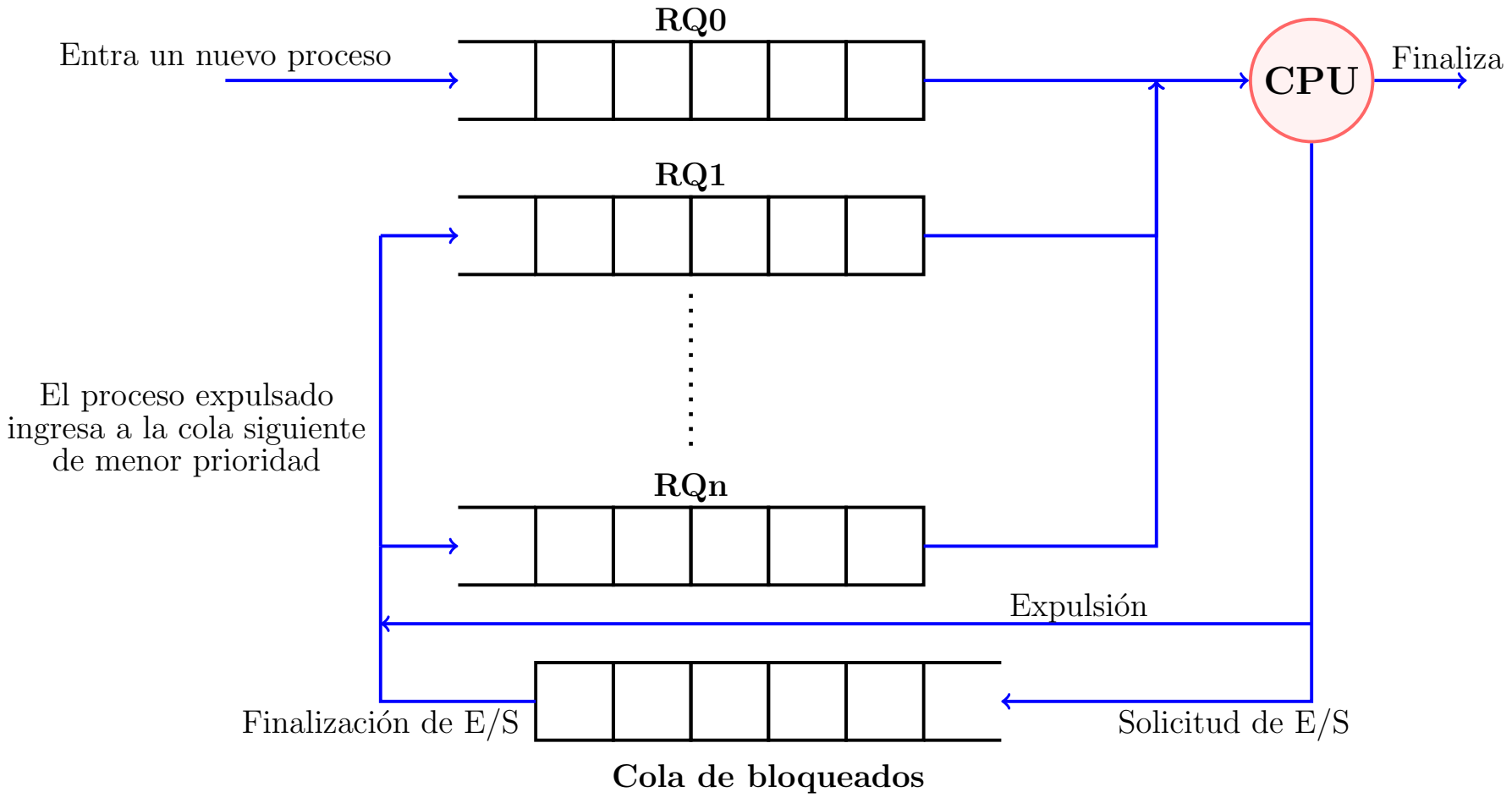
- **Ventaja:** Permite definir claramente la importancia relativa de cada proceso.
- **Desventaja:** La planificación por prioridades puede generar **inanición** (*starvation*) de los procesos de baja prioridad. Esto ocurre cuando hay siempre procesos de mayor prioridad disponibles, de modo que los procesos de menor prioridad nunca llegan a ejecutarse.

Para mitigar este problema, se pueden implementar mecanismos de **envejecimiento** (*"aging"*): la prioridad de un proceso de baja prioridad aumenta gradualmente mientras espera en la cola de listos, hasta que eventualmente alcanza un nivel suficiente para ser ejecutado.

Colas multinivel con realimentación (MLFQ, *Multilevel Feedback Queue*)

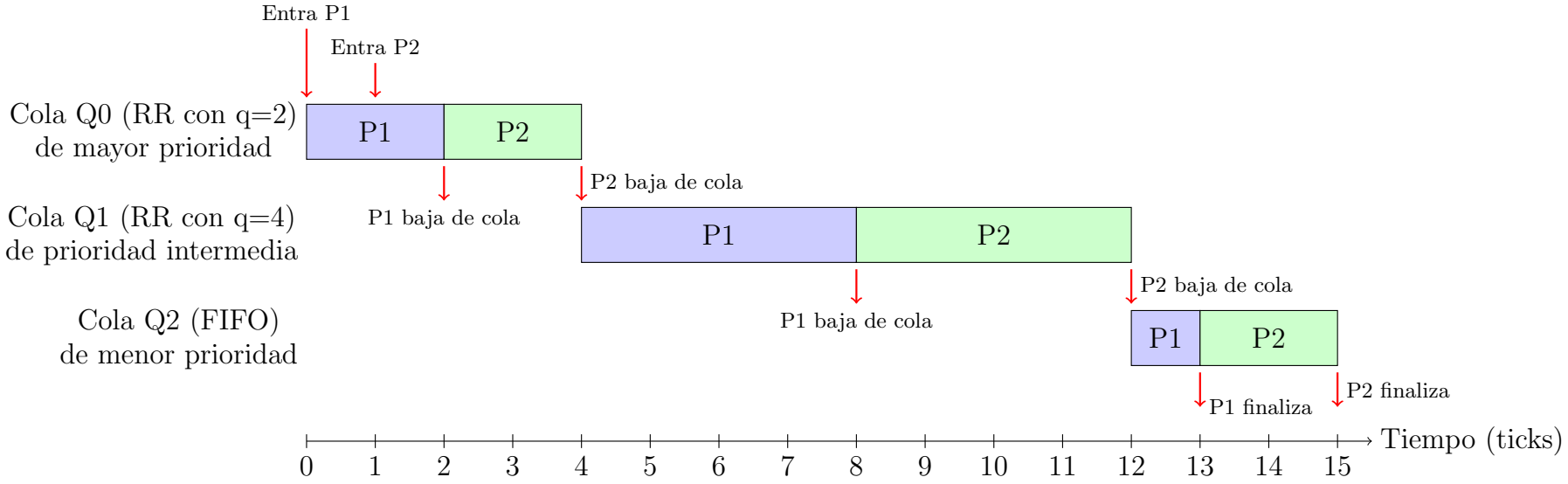
- **MLFQ** organiza los procesos en **múltiples colas de prioridad**.
- Cada cola puede usar una política distinta: **FCFS** o **Round-Robin**.
- La cola de mayor prioridad se atiende primero; solo si está vacía se atiende la siguiente.
- En una cola FCFS, los procesos se ejecutan en orden de llegada.
- En una cola RR, un proceso que consume todo su quantum desciende a la cola inferior.
- Los procesos cortos terminan rápido en las colas superiores.
- Los procesos largos descienden gradualmente de cola, respetando el orden en cada una.

Colas multinivel con realimentación (MLFQ, *Multilevel Feedback Queue*)



Colas multinivel con realimentación (MLFQ, *Multilevel Feedback Queue*)

- Los procesos nuevos comienzan en la cola de más alta prioridad.
- Cuando el proceso consume completamente su quantum sin bloquearse ni finalizar, se lo degrada a una cola de menor prioridad.
- Cuando un proceso sale del estado de bloqueado, retorna a la misma cola de la que fue desalojado.



Colas multinivel con realimentación (MLFQ, *Multilevel Feedback Queue*)

- Para evitar la "inanición", los procesos que esperan demasiado pueden ser promovidos a colas de mayor prioridad (*feedback*), ya sea mediante reglas explícitas de reinserción o incrementando gradualmente su prioridad (*aging*).
- **MLFQ** favorece a los procesos cortos, ya que estos terminarán sus tareas sin ser relegados a prioridades inferiores.
- **Ventajas:** Es eficiente y flexible.
- **Desventaja:** Complejidad.

Lotería

- Cada proceso tiene un número determinado de boletos; cada boleto representa una oportunidad de ganar la lotería.
- Cada vez que el planificador debe elegir el siguiente proceso a ejecutar, selecciona un número al azar y asigna el siguiente *quantum* al proceso que posee el boleto ganador.
- **Tipo:** Apropiativo, ya que el proceso en ejecución puede ser desalojado en cada nueva planificación.
- El boleto "ganador" no se retira; es decir, la probabilidad de que un proceso sea seleccionado no cambia entre invocaciones sucesivas del planificador.
- Un proceso al que se le quiera dar mayor prioridad simplemente recibe más boletos. Por ejemplo, si el proceso A tiene 20 boletos y el proceso B tiene 60, es tres veces más probable que el siguiente turno corresponda a B que a A.

Lotería

- **Ventajas:**
  - Implementación simple.
  - Permite asignar prioridades de forma proporcional (más boletos $\Rightarrow$ más CPU).
  - Evita la inanición: todo proceso con al menos un boleto tiene probabilidad de ejecutarse.
- **Desventajas:**
  - No es determinista: no garantiza tiempos de respuesta acotados en el corto plazo.
  - Puede presentar variabilidad (alta dispersión) en la asignación de CPU.
  - Requiere un generador de números aleatorios adecuado.
  - Menor previsibilidad que algoritmos como *Round Robin* o prioridades fijas.

Clasificación de los mecanismos de planificación a corto plazo

- Los algoritmos presentados pueden clasificarse según dos criterios principales:
  1. Si están pensados para multitarea cooperativa o apropiativa.
  2. Si emplean información sobre los procesos.

	No necesita información del proceso	**Necesita información del proceso**
Cooperativa	FCFS	SPN HPRN
Apropiativa	RR MLFQ Lotería	PSPN SRR Prioridades

Evaluación de algoritmos de planificación

- El comportamiento de los algoritmos depende de la carga de trabajo:
  - Duración de los procesos
  - Frecuencia de llegada
  - Mezcla de procesos de diferentes tipos (CPU-bound, I/O-bound)
- Un único experimento no es representativo.
- Es necesario realizar múltiples simulaciones:
  - Con distintas cargas de trabajo:

$$\rho = \frac{\lambda}{\mu}$$

    - λ: tasa de llegada de procesos (procesos por unidad de tiempo).
    - μ: tasa de servicio (cantidad de procesos que el sistema puede completar por unidad de tiempo).
  - Con variabilidad controlada (aleatoriedad)
- Las conclusiones deben basarse en resultados estadísticos.

Bibliografía

Silberschatz, A., Galvin, P. B., and Gagne, G. (2018).
Operating System Concepts Essentials.
Wiley Publishing.

Stallings, W. (2011).
Operating Systems: Internals and Design Principles.
Prentice Hall Press.

Tanenbaum, A. S. (2009).
Sistemas Operativos Modernos.
Pearson Educación.

Wolf, G., Ruiz, E., Federico, B., and Erwin, M. (2015).
Fundamentos de Sistemas Operativos.
Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas : Facultad de Ingeniería.